UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
INTELIGÊNCIA ARTIFICIAL
TURMA 01

CARLOS ARTUR ALENCAR CRUZ
GUILHERME INACIO SANTOS PAES
IRWING FELIPE PEREIRA VIEIRA
RAPHAEL MATOS DA SILVA GONÇALVES
RENAN NUNES ANDRADE SAMPAIO

LLM

SÃO CRISTÓVÃO - SE

22/02/2026

INTRODUÇÃO

Este relatório tem como objetivo mostrar a implementação de uma LLM baseada na API do Gemini 2.5-Flash. O projeto desenvolvido consiste em um "Gerador de Receitas com Restrições Alimentares", projetado para atuar como um assistente virtual capaz de reter informações passadas pelo usuário através de um sistema de memória conversacional. O desenvolvimento foi pautado pelas diretrizes e pseudocódigos propostos para o "Cenário A" (Chatbot com memória).

DEFINIÇÃO DO PROBLEMA

O foco central do projeto foi contornar o maior desafio lógico em implementações de chat puro sem o uso de banco de dados: a manutenção do histórico de conversa. APIs de LLMs, por natureza, não possuem estado (são *stateless*), o que exige que todo o contexto da conversa seja reenviado a cada nova interação.

O problema principal consistia em garantir que o assistente lembrasse as alergias alimentares relatadas pelo usuário nas interações iniciais e aplicasse essa restrição na geração de receitas futuras, tudo isso sem ultrapassar o limite máximo de tokens estabelecido (Gestão de Contexto)

MODELAGEM

A solução foi desenvolvida em Python, com interface provida pela biblioteca Streamlit, utilizando o SDK google.gena, seguindo, também, como base o PDF passado pelo Professor Hendirk. A modelagem do projeto foi estruturada em três frentes:

- **Configuração de Hiperparâmetros (MODEL-CONFIG):** Para garantir consistência gastronômica e evitar "alucinações" que poderiam gerar riscos de segurança alimentar para um usuário alérgico, a **temperatura** (nível de criatividade) do modelo foi calibrada para 0.7. Também foi implementada uma instrução de sistema (*System Prompt*) configurando a IA como um "chef especialista em receitas adaptadas".
- **Gestão de Estado:** O vetor de histórico de mensagens foi encapsulado no st.session_state do Streamlit. Isso evita que as variáveis sejam resetadas a cada interação (recarregamento da página), simulando uma memória contínua. As mensagens foram tipadas estritamente com os papéis (role) exigidos pela API: user para as perguntas do usuário e model para as respostas da IA.
- **Truncamento de Contexto:** Para evitar o estouro de limite de 4096 tokens, aplicou-se a lógica de verificação de tamanho da lista de mensagens. Caso a conversa ultrapasse o limite estimado (4 caracteres $\simeq$ 1 token), a estrutura descarta progressivamente os pares de interação (pergunta e resposta) mais antigos, liberando espaço para as novas requisições sem quebrar a alternância obrigatória de papéis exigida pelo Gemini.

PSEUDOCÓDIGO E EQUIVALÊNCIA COM O CÓDIGO IMPLEMENTADO:

A seguir, veja a como o código foi construído, seguindo como base o pseudocódigo presente no PDF:

```
function CONVERSATIONAL-AGENT(user_input, history, config) returns a response, updated_history
    // 1. Construção da Mensagem
    current_msg CREATE-MESSAGE(role="user", content=user_input)
    APPEND(history, current_msg)

    // 2. Gestão de Contexto (Obrigatório para não estourar tokens)
    // Mantém o System Prompt fixo e remove mensagens antigas se necessário
    payload TRUNCATE-HISTORY(history, limit=config.max_tokens)

    // 3. Chamada de Inferência (Texto)
    api_response CALL-LLM-API(config, payload)

    // 4. Pós-processamento
    reply_text EXTRACT-CONTENT(api_response)
    reply_msg CREATE-MESSAGE(role="assistant", content=reply_text)
    APPEND(updated_history, reply_msg)

    return reply_text, updated_history
```

```python
# Aplicação do cenário A adaptado ao uso de memória para as alergias:
def CONVERSATIONAL_AGENT(user_input, history, config):  1 usage  IrwingVieira +1 *

    # Construção da Mensagem e atualização do histórico:
    current_msg = CREATE_MESSAGE(role="user", content=user_input)
    history.append(current_msg)

    # Gestão de Contexto (Simplificado aqui)
    payload = TRUNCATE_HISTORY(history, limit=4096)

    # Chamada da API
    api_response = CALL_LLM_API(config, payload)

    # Pós-processamento da IA.
    reply_text = EXTRACT_CONTENT(api_response)
    reply_msg = CREATE_MESSAGE(role="model", content=reply_text)

    #Guarda o resultado do processamento no histórico.
    history.append(reply_msg)

    return reply_text, history
```

Para melhor visualização do que foi feito, segue o link para o repositório: *https://github.com/arturalencar/aplicacao-LLM*